REZON · how it works

Every part of the fall detector: what it says to the next part, where the current flows, and the maths behind it

2026-11-12
final build: SM-24 + ESP32-S3 + SIM868

THE ONE-MINUTE VERSION

A fall shakes the floor. A **geophone** (a magnet and a coil on a spring) turns that shaking into a few millivolts. A **24-bit ADC** measures the voltage 1000 times a second and hands the numbers to the **ESP32-S3** over SPI. The ESP32 filters the signal, tells footsteps from impacts, and watches what happens *after* an impact. Impact followed by no walking means a likely fall: the **speaker** asks “Mari néni, jól van?”, the **LED** changes colour, and if nobody presses the **“I’m OK” button**, the **SIM868 modem** phones for help. One 18650 cell powers everything.

WHERE EACH NUMBER IN THIS DOCUMENT COMES FROM

MEASURED our own bench results from 12 Nov (recordings in `uploads/`, call test in `NOTES.md`). DESIGN taken from our wiring source, parts list and test sketches. TYPICAL datasheet or textbook values I have not measured on our unit; check before quoting them as ours.

Two things to know: the **detection logic** is described as it runs today in the tuning tool (`practice/geo_viz`); the firmware on the device should use the same numbers. The **pin numbers** are the final ESP32-S3 map; the practice rig on the classic ESP32 uses different pins.

1 The whole system at a glance

There are three chains that meet in the ESP32-S3: a **sensing chain** coming in, an **alert chain** going out, and the **power chain** underneath both.

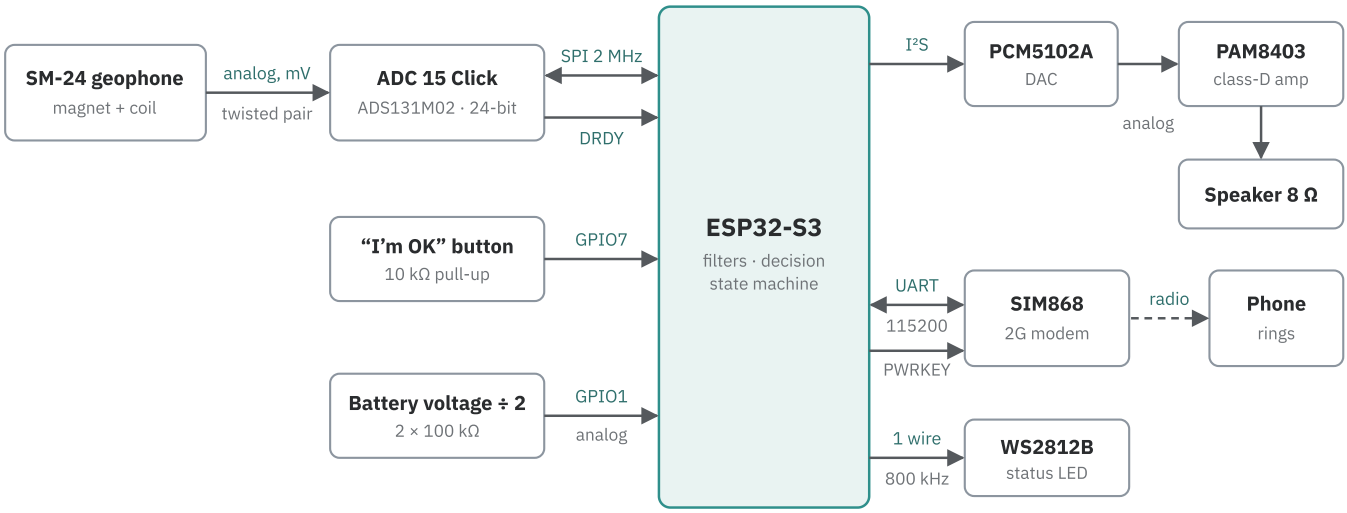


Figure 1. Who talks to whom. Arrows show the direction of information; double arrows carry data both ways.

LINK	KIND OF SIGNAL	WIRES	SPEED	WHAT TRAVELS ON IT
Geophone → ADC	Analog voltage	2 (twisted pair)	continuous	A voltage proportional to how fast the floor moves
ADC ↔ ESP32	SPI (digital, clocked)	4 + DRDY + reset	2 MHz clock, 1000 frames/s	Two 24-bit samples and a checksum per frame; settings the other way
ESP32 → clock chip	SPI, own chip-select	shares the SPI wires	once at start-up	One 16-bit word that sets the ADC’s 8.192 MHz clock
ESP32 → DAC	I²S (digital audio)	3	about 1.4 MHz bit clock	The voice clip as 16-bit sound samples
DAC → amp → speaker	Analog audio, then switched power	2 + 2	audio band	The same clip as a voltage, then as current through the speaker coil

LINK	KIND OF SIGNAL	WIRES	SPEED	WHAT TRAVELS ON IT
ESP32 ↔ SIM868	UART (digital, no clock wire)	2 + PWRKEY	115 200 bit/s	Text commands (“AT…”) and text replies
SIM868 ↔ mobile network	Radio, GSM 900/1800 MHz	antenna	bursts every 4.6 ms	The phone call
ESP32 → LED	One-wire pulse code	1	800 000 bit/s	24 bits: green, red, blue brightness
Button → ESP32	On/off level	1	–	3.3 V = not pressed, 0 V = pressed

Five ideas that explain every wire

- **Voltage** (volts) is the push; **current** (amps) is the flow of charge; **resistance** (ohms) is how hard a part resists the flow. Ohm’s law ties them: $V = I \cdot R$.
- **Power** is $P = V \cdot I$ (watts). Energy is power × time; a battery stores energy in watt-hours.
- **Current flows in loops.** Whatever leaves the battery’s + terminal must return to its – terminal. That return path is **ground (GND)**, which is why every module has a GND wire and why they all meet on one rail.
- A **digital signal** is just a voltage that is either near 0 V (“0”) or near the supply (“1”). Almost no current flows on a signal wire; it only charges the tiny input capacitance of the receiver.
- A **capacitor** is a small local reservoir of charge: $Q = C \cdot V$. It supplies short bursts faster than a battery at the end of a wire can.

2 Power: where the current flows

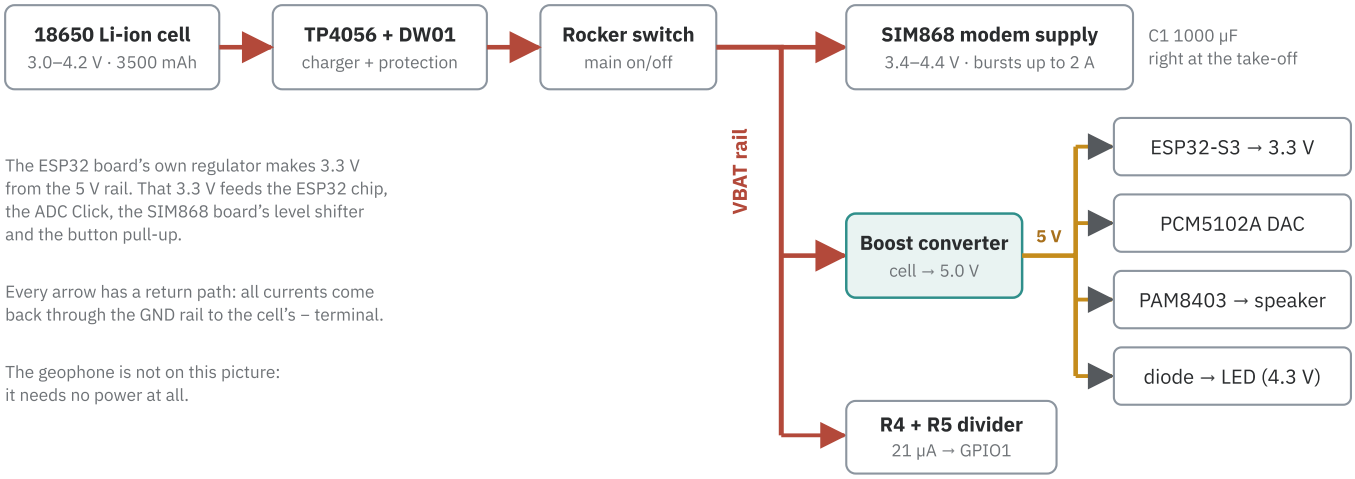


Figure 2. The power tree. Red = battery voltage, amber = 5 V. DESIGN

From the cell to the two rails

- **18650 cell.** 4.2 V full, about 3.6 V for most of its discharge, 3.0 V empty. Stored energy: 3.6 V × 3.5 Ah ≈ **12.6 Wh**.
- **TP4056 + DW01.** When USB-C is plugged in, the TP4056 charges the cell: constant current first (1 A with the module's standard resistor), then it holds 4.2 V while the current tapers off. The DW01 chip and its two MOSFETs sit in the cell's – lead and open the circuit on over-discharge (about 2.4 V), over-charge or a short (about 3 A). TYPICAL
- **Rocker switch.** Simply breaks the + line. It was chosen because it survives the modem's 2 A bursts.
- **VBAT rail** (bare wire along column 35). Two heavy users hang on it: the modem directly, and the boost converter.
- **5 V rail** (row Y), buffered by C3 = 470 µF.

Why the modem gets the raw battery and nothing else does

The SIM868 is built to run from a single lithium cell (3.4–4.4 V). 5 V would destroy it, so header pins 39 and 40 on its board stay empty. Everything else wants a steady 5 V or 3.3 V, which a discharging cell cannot give, so a boost converter lifts the cell voltage to 5.0 V.

The boost converter

A switch connects an inductor across the cell for a fraction *D* of each cycle. Current builds in the inductor and stores energy in its magnetic field. When the switch opens, the inductor keeps the current flowing and pushes it through a diode into the output at a higher voltage.

> $V_{out} = V_{in} / (1 - D) \implies D = 1 - V_{in}/V_{out} = 1 - 3.7/5.0 = 0.26$ the switch is on 26 % of the time at 3.7 V

Energy is conserved, so a higher voltage out means a higher current in:

> $I_{in} = V_{out} \cdot I_{out} / (\eta \cdot V_{in}) = 5.0 \times 0.30 / (0.9 \times 3.7) \approx 0.45\ \text{A}$ for 0.3 A on the 5 V rail, efficiency η ≈ 90 % (typical)

The modem's 2 A bursts and capacitor C1

GSM is a time-shared radio: a phone transmits in one of eight time slots. During a call the modem's transmitter switches on for 577 µs, then is off for seven slots, 217 times a second.

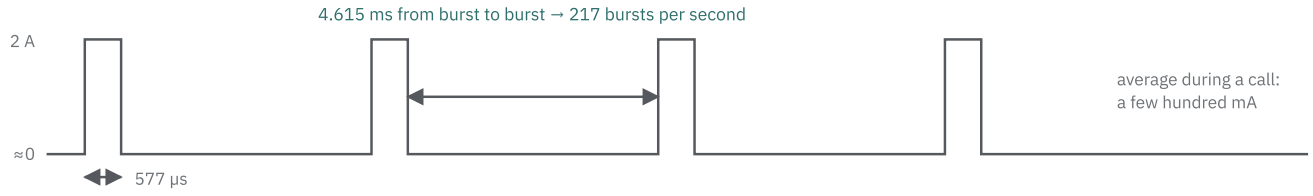


Figure 3. Supply current of the modem while it transmits. TYPICAL

Every wire, the switch, the protection MOSFETs and the cell itself have a little resistance. If the whole path adds up to about 0.15 Ω, a 2 A burst drops the voltage at the modem by

> $\Delta V = I \cdot R = 2\ \text{A} \times 0.15\ \Omega = 0.3\ \text{V}$ 0.15 Ω is my estimate, not a measurement

If the cell is at 3.6 V, the modem sees 3.3 V for an instant, which is under its 3.4 V minimum, and it may reset in the middle of dialling. Three things protect us: **short, thick (0.5 mm²) leads** keep *R* low; a **well-charged cell** keeps the starting voltage high; and **C1 = 1000 µF** right at the take-off delivers the very first rush of each burst. C1 alone could not carry a burst (2 A for 577 µs would pull it down by $\Delta V = I \cdot t/C = 1.15$ V); its job is to take the sharp edge off, so the battery only has to follow more gently.

Who draws how much

CONSUMER	RAIL	CURRENT	NOTE
ESP32-S3 (radio off)	5 V → 3.3 V	about 40–80 mA	The on-board regulator turns the extra 1.7 V into heat TYPICAL
ADC 15 Click	3.3 V	a few mA	ADC plus its clock chip TYPICAL
PCM5102A DAC	5 V	about 20 mA	TYPICAL
PAM8403 + speaker	5 V	about 10 mA idle, up to about 240 mA at 1 W	Only loud while the question plays
WS2812B pixel	4.3 V	1 mA dark, up to 60 mA full white	About 20 mA per colour at full brightness TYPICAL
SIM868	VBAT	about 20 mA idle, bursts to 2 A in a call	TYPICAL
Battery divider	VBAT	21 µA	4.2 V / 200 kΩ
Button pull-up	3.3 V	0.33 mA, only while pressed	3.3 V / 10 kΩ

Battery life, rough estimate: about 0.1 A on the 5 V rail is 0.5 W, or 0.57 W from the cell after the boost; add about 0.07 W for the idle modem. 12.6 Wh / 0.64 W ≈ **20 hours** of watching. This is an estimate from typical figures; we have not measured it.

Three small circuits on the power side

Battery voltage divider (R4, R5). An ESP32 pin must never see more than 3.3 V, and a full cell is 4.2 V. Two equal resistors in series split the voltage in half:

> $V_{\text{GPIO1}} = V_{\text{BAT}} \cdot \text{R5}/(\text{R4}+\text{R5}) = 4.2 \times ½ = 2.1\ \text{V}$

The ESP32's analog input reads that and the code doubles it. The modem can also report its own supply with `AT+CBC` (it said 3.99 V in our test MEASURED).

Diode in the LED supply (1N4001). A WS2812B only accepts a "1" if the data voltage is above 0.7 × its supply. At 5 V that is 3.5 V, more than the ESP32's 3.3 V. A silicon diode drops about 0.7 V, so the LED runs on 4.3 V and the limit becomes

> $0.7 \times 4.3\ \text{V} = 3.0\ \text{V} < 3.3\ \text{V}$ ✓

100 nF capacitors beside each module do for every chip what C1 does for the modem: a tiny local reservoir for fast switching currents.

3 The geophone: floor motion into voltage

The SM-24 has no electronics and needs no power. Inside the case a strong magnet is fixed to the housing; around it hangs a coil of fine wire on two soft springs. When the floor moves, the case and the magnet move with it. The coil, because of its inertia, tries to stay where it was. So the magnet moves *relative to the coil*, and a moving magnetic field induces a voltage in a coil.

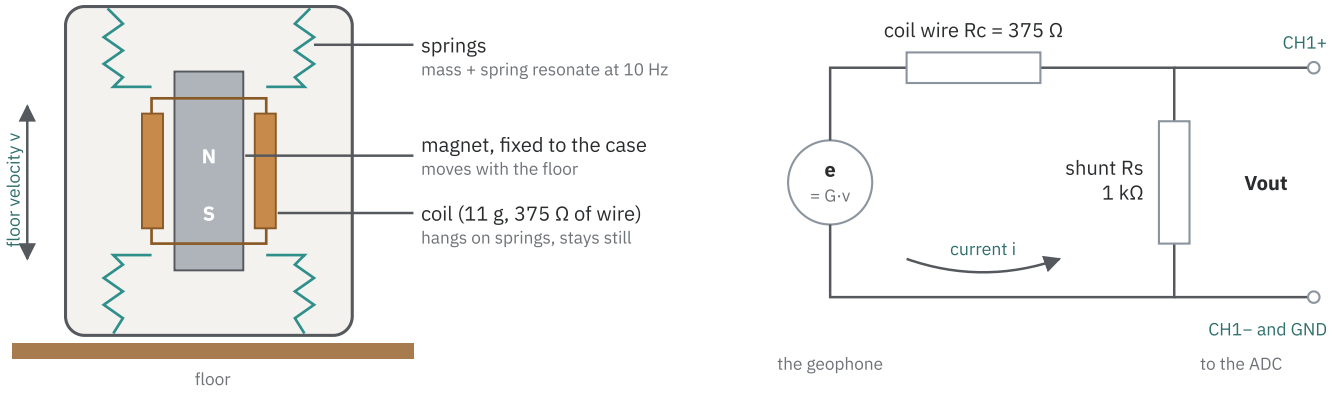


Figure 4. Left: inside the SM-24. Right: the same thing as a circuit: a voltage source with the coil's own wire resistance, loaded by our 1 kΩ resistor.

Faraday's law gives the voltage

$$e = B \cdot l \cdot v = G \cdot v, \quad G = 28.8 \text{ V per m/s}$$ B: magnetic field, l: length of wire in the field, v: coil-to-magnet velocity TYPICAL

The output is proportional to **velocity**, not to position or acceleration. A floor moving at 1 mm/s gives 28.8 mV. No battery is involved: the energy comes from the floor's motion.

The 1 kΩ resistor: a deliberate load

With the resistor across the terminals, current flows around the loop in Figure 4, and the ADC sees only the part of *e* that falls across the resistor (a voltage divider with the coil's own 375 Ω):

$$i = e \,/\, (R_c + R_s) \quad V_{out} = e \cdot R_s/(R_c + R_s) = e \times 1000/1375$$

$$G_{loaded} = 28.8 \times 0.727 = \mathbf{20.9 \text{ V per m/s}} = 20.9 \text{ mV per mm/s}$$ the number in our software

Why give away 27 % of the signal? Because of **damping**. A mass on a spring rings like a bell after every knock. The current *i* in the coil creates a force $F = G \cdot i$ that opposes the motion (Lenz's law): an electromagnetic brake. More current, more braking. The damping ratio ζ tells how quickly the ringing dies (0 = rings forever, 1 = no overshoot at all):

$$\zeta = \zeta_{open} + G^2 \,/\, (2 \cdot m \cdot \omega_0 \cdot (R_c + R_s)) = 0.25 + 28.8^2 \,/\, (2 \times 0.011 \times 62.8 \times 1375) = 0.25 + 0.44 \approx \mathbf{0.69}$$

With ζ ≈ 0.7 the coil settles almost at once and the response is flat above 10 Hz, so one footstep looks like one clean burst instead of a long wobble. TYPICAL (computed from datasheet values: 11 g moving mass, $\omega_0 = 2\pi \times 10$ Hz.)

How it responds at each frequency

The coil on its springs is a second-order system. Its output for a floor vibrating at frequency *f* is

$$|H(f)| = r^2 \,/\, \sqrt{((1 - r^2)^2 + (2\zeta r)^2)}, \quad r = f \,/\, 10 \text{ Hz}$$

FREQUENCY	5 HZ	10 HZ	15 HZ	20 HZ	40 HZ AND ABOVE
Share of full sensitivity	25 %	73 %	93 %	98 %	100 %

Below 10 Hz the coil simply rides along with the case and the output falls away quickly. That is fine: footsteps and impacts live between about 10 and 60 Hz.

How small these motions are

For a vibration at frequency *f*, displacement = $v \,/\, (2\pi f)$ and acceleration = $2\pi f \cdot v$.

EVENT	VELOCITY	GEOPHONE VOLTAGE	FLOOR DISPLACEMENT	CURRENT IN THE COIL
Smallest step we accept (threshold)	0.02 mm/s	0.42 mV	about 0.1 µm at 30 Hz	0.4 µA
Normal footstep nearby MEASURED	about 0.1 mm/s	2.1 mV	about 0.5 µm at 30 Hz	2 µA
Impact threshold	0.3 mm/s	6.3 mV	about 3 µm at 15 Hz	6 µA
Hardest hit in our recordings MEASURED	3.9 mm/s	82 mV	about 42 µm at 15 Hz	82 µA

A human hair is about 70 µm thick. The device notices floor movements several hundred times smaller than that.

The cable

The two geophone wires are **twisted together**. Any stray magnetic field (mains wiring, the modem's bursts) induces a voltage in a loop of wire in proportion to the loop's area. Twisting makes the loop area tiny and flips its direction every turn, so what one half-turn picks up the next one cancels. The ADC also measures the *difference* between CH1+ and CH1−, so anything that lands equally on both wires disappears.

4 The ADC: voltage into numbers

The ADC 15 Click carries a Texas Instruments **ADS131M02**: two channels, 24 bits each, sampled at the same instant. Our geophone is on channel 1. Channel 2 is shorted, so whatever it reads is the electronics' own noise: a built-in reference for "how quiet is quiet".

Gain and full scale

The chip compares the input with an internal 1.2 V reference, after a built-in amplifier with a selectable gain. The range it can measure is ±1.2 V / gain.

GAIN	VOLTAGE RANGE	VELOCITY RANGE (÷ 20.9 MV PER MM/S)	VERDICT
× 32	± 37.5 mV	± 1.8 mm/s	Clipped: our stomps reached 6.6 mm/s MEASURED
× 4	**± 300 mV**	**± 14.3 mm/s**	Current setting: room for the hardest fall

The chip can read inputs slightly below ground, so the geophone's swing around 0 V needs no bias circuit: CH1− is simply tied to GND. TYPICAL

From counts to millimetres per second

A 24-bit signed number runs from −8 388 608 to +8 388 607 (that is 2^{23}). One count is therefore

> $1\ \text{count} = 1.2\ \text{V} / (\text{gain} \times 2^{23}) = 1.2 / (4 \times 8\,388\,608) = \mathbf{35.8\ nV}$

> $v\ [\text{mm/s}] = \text{counts} \times 35.8\ \text{nV} / 20.9\ \text{mV per mm/s} = \text{counts} \times 1.71 \times 10^{-6}$ one count = 1.7 nanometres per second

So the smallest step we accept (0.02 mm/s) is about 11 700 counts and the impact threshold (0.3 mm/s) is about 176 000 counts. The resolution is far finer than we need; what limits us is the room's own vibration, not the ADC.

How 24 bits are made: delta-sigma

Inside, the ADC is surprisingly crude: a 1-bit comparator running very fast (4.096 million decisions per second), wrapped in a feedback loop that makes the *average* of its 1s and 0s follow the input. A digital filter then averages 4096 of these decisions into each output sample. Averaging many coarse readings gives one fine one, and it removes everything above the wanted band, which is why no separate filter is needed in front of the ADC.

> $8.192\ \text{MHz clock} \div 2 = 4.096\ \text{MHz modulator} \rightarrow \div 4096 = \mathbf{1000\ samples\ per\ second}$

With 1000 samples per second we can represent signals up to 500 Hz (the Nyquist limit: half the sample rate). We only use 10–60 Hz, so there is a wide safety margin.

The clock chip

The ADC has no clock of its own. A second chip on the Click, the **LTC6903**, is a programmable oscillator. At start-up the ESP32 sends it one 16-bit word (`0xCF62`) over the same SPI wires, using a second chip-select line:

> $f = 2^{\text{OCT}} \times 2078\ \text{Hz} / (2 - \text{DAC}/1024) = 2^{12} \times 2078 / (2 - 984/1024) = 8.1915\ \text{MHz}$

Without this step the ADC stays silent, which is the first thing to check if no data arrives.

SPI: how the samples reach the ESP32

SPI is a conversation on four wires, always led by the ESP32:

WIRE	ESP32-S3 PIN	DIRECTION	JOB
SCK	GPIO12	ESP32 → ADC	The clock: one tick per bit, 2 million ticks per second
MOSI (SDI)	GPIO14	ESP32 → ADC	Commands and settings
MISO (SDO)	GPIO13	ADC → ESP32	Samples and answers
CS1	GPIO11	ESP32 → ADC	"I am talking to you": low for the length of one frame
CS2	GPIO9	ESP32 → clock chip	The same, for the LTC6903
DRDY (INT)	GPIO10	ADC → ESP32	Drops low when a new sample is ready: triggers an interrupt

WIRE	ESP32-S3 PIN	DIRECTION	JOB
RST	GPIO8	ESP32 → ADC	Pulled low for 2 ms at start-up to reset the ADC

On every clock tick, one bit travels in *each* direction at the same time. The ADC changes its output on one clock edge and the ESP32 reads it on the other (SPI “mode 1”), so the data is always stable when it is read.

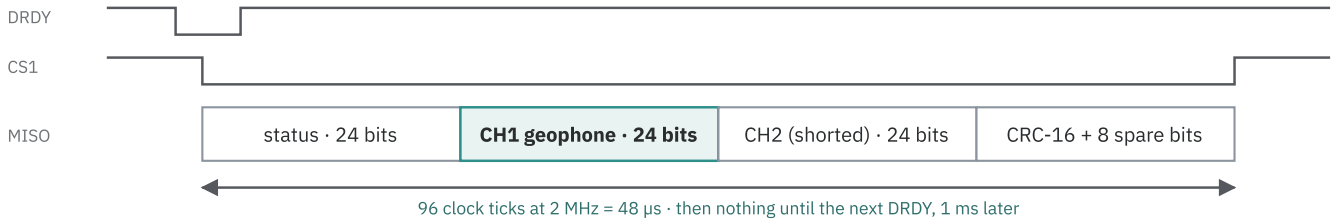


Figure 5. One SPI frame. The ADC is busy talking for less than 5 % of the time.

The CRC is a 16-bit checksum the ADC computes over the first nine bytes. The ESP32 computes the same thing (polynomial 0x1021, start value 0xFFFF) and compares. One flipped bit anywhere in the frame gives a different result, so a glitch on the wires can never pass as a real vibration.

Settings go the other way in the first word of a frame: a write-register command followed by the value. For example, the clock register gets `0x0316` (both channels on, averaging 4096, high-resolution mode) and the gain register gets `0x0022` (× 4 on both channels). The ADC’s answer to a read arrives one frame later.

5 The maths: from samples to “this is a fall”

All of this runs for every single sample, 1000 times a second. Each step is a few multiplications.

Step 1: remove the offset

The ADC has a small constant offset. A slow running average follows it, and we subtract that average:

> $\mathrm{dc} \leftarrow \mathrm{dc} + 0.002 \cdot (x - \mathrm{dc}), \quad y = x - \mathrm{dc}$ a one-pole high-pass; corner ≈ 0.002 × 1000 / 2π ≈ 0.3 Hz

Step 2: two band-pass filters

The same signal goes down two paths, each keeping only the frequencies where its target is strongest:

PATH	BAND	WHY THIS BAND
Impact path	10–45 Hz	A body hitting the floor puts most of its energy below 20 Hz, and it is 20–50 times stronger than any background. Stops short of the 50 Hz mains hum.
Step path	18–60 Hz + hum canceller	Our test building hums at 14–15.3 Hz. Several close tones “beat” into slow swells of 0.04–0.07 mm/s that look like footsteps. Starting at 18 Hz avoids most of it. MEASURED

Each filter is built from four “biquad” sections (two high-pass, two low-pass), giving a 4th-order Butterworth edge on each side: flat inside the band, falling by 24 dB per octave outside. One biquad computes each output from the current input, the two previous inputs and the two previous outputs:

> $y[n] = b_0 x[n] + b_1 x[n-1] + b_2 x[n-2] - a_1 y[n-1] - a_2 y[n-2]$

The five coefficients come from the corner frequency f_0 and a sharpness value Q ($\omega = 2\pi f_0/1000$, $\alpha = \sin\omega / 2Q$). The two Q values, 0.5412 and 1.3066, are what make a pair of biquads into a Butterworth filter.

Step 3: the hum canceller (step path only)

A steady tone is predictable: if you know the last 50 ms, you know the next sample. A footstep is not. So a small adaptive filter tries to **predict** each sample from 48 samples that are at least 25 ms old, and we keep only the part it *failed* to predict:

> $\text{prediction } p = \Sigma\, w_i \cdot x[n - 25 - i], \quad \text{error } e = x[n] - p, \quad w_i \leftarrow w_i + 0.004 \cdot e \cdot x[n - 25 - i] \,/\, (\text{signal power})$

The weights w learn the hum by themselves and keep tracking it if it drifts. The error e is the output: hum removed, footsteps intact. In our recordings this lifted the weakest steps from 0.29× to 0.90× the size of the strongest hum swell. MEASURED

Step 4: the envelope

A vibration swings positive and negative; we want its *size*. Squaring makes everything positive, a 15 ms running average smooths it, and the square root brings back the units:

> $m \leftarrow m + (y^2 - m)/15, \quad \text{envelope} = \sqrt{(2m)}$ √2 turns the RMS of a sine into its peak

Step 5: a threshold that adapts to the room

A fixed threshold fails as soon as the room gets noisier. So we track the **background level** with a follower that drops quickly but rises slowly (time constants 0.1 s down, 2 s up). It settles on the quiet moments between events and ignores the events themselves.

> $\text{step threshold} = \max(0.02 \text{ mm/s}, 2 \times \text{background})$

A **hit** starts when the step envelope crosses this threshold (or the impact envelope gets near its own level), and ends when both envelopes fall to 60 % of their peak. Hits closer together than 0.25 s are merged.

Step 6: step, walking or impact?

- **Impact:** the hit’s peak in the 10–45 Hz path is at least **0.3 mm/s**.
- **Step:** any smaller hit. Two hits less than 0.35 s apart count as one footfall (heel and toe).

- **Walking:** three or more footfalls, each 0.3–1.6 s after the last. Cadence = 120 / (sum of the last two gaps) steps per minute.
- **Rhythm tracking:** once the cadence is steady, the next step is *expected*. Within ±25 % of the expected moment the threshold drops to 60 %, so soft or distant steps are still caught.

Step 7: what makes it a fall

An impact alone proves nothing: a dropped bag makes one too. The difference is **what comes next**.

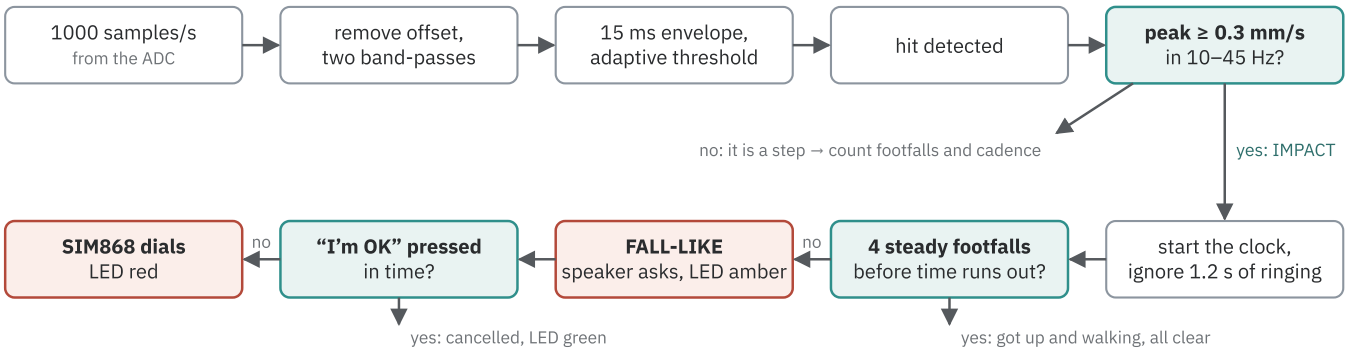


Figure 6. The decision chain. Top row: signal processing. Bottom row: the logic after an impact.

- After an impact a clock starts. The first 1.2 s is ignored, because the floor is still ringing.
- The only thing that cancels the alarm is **walking**: 4 footfalls in a row, each 0.35–1.6 s apart. Shuffling, trying to push up, or other people far away do *not* cancel it.
- If the clock runs out first, it is **FALL-LIKE**. The tuning tool uses 5 s; the stage plan is to ask the question straight away and wait 3 s for the button. Both are single numbers in the code. DESIGN

Tuned on our recordings of 12 Nov: about 97 % of steps found, about 0.2 false hits per second while lying still, and all 7 known falls flagged. MEASURED

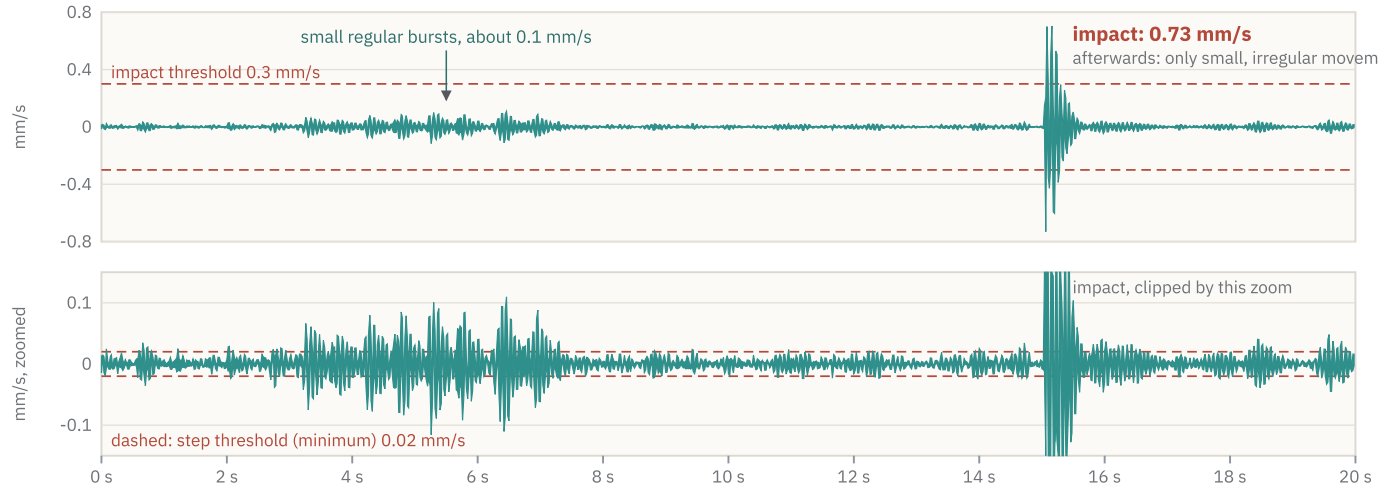


Figure 7. Twenty seconds of our own data (recording of 12 Nov, 10:50, band-pass filtered, min/max per 20 ms). Top: full scale. Bottom: the same trace zoomed in 5×, where the small bursts and the background become visible. The recording carries no labels; reading the small bursts as footsteps and the spike as a test fall is my interpretation of it. MEASURED

This picture is the whole idea in one view: the impact is roughly seven times larger than anything before it, so a single threshold finds it; and what follows it (no regular rhythm of walking, only small irregular movement) is what turns “impact” into “fall”.

6 The alert chain: button, light, voice, phone call

The “I’m OK” button

A 10 kΩ resistor (R1) connects GPIO7 to 3.3 V, so the pin normally reads “1”. The button connects the same pin to GND. Pressed, the pin reads “0”, and 3.3 V / 10 kΩ = 0.33 mA flows through the resistor. Without the pull-up resistor the open pin would float and read random values. Mechanical contacts bounce for a few milliseconds, so the code should only accept a press that stays low for 20–30 ms.

The status LED (WS2812B)

One data wire (GPIO15) carries all three colours. Each pixel has a tiny chip that reads 24 bits (8 each for green, red, blue) and drives its three LEDs with those brightness values. There is no clock wire, so a bit’s value is carried by **how long the pulse stays high** within a fixed 1.25 µs slot:

> “0” = high 0.4 µs, low 0.85 µs · “1” = high 0.8 µs, low 0.45 µs · 24 bits = 30 µs TYPICAL

The pixel holds the colour until new data arrives. Green = watching, amber = asking, red = calling.

The voice: I²S → DAC → amplifier → speaker

The recorded question is stored in the ESP32’s flash as a list of numbers: 16-bit samples, tens of thousands per second. Getting it to your ear takes four conversions.

1. **I²S (digital, 3 wires).** BCK (GPIO4) ticks once per bit; LCK (GPIO5) flips between left and right channel, once per sample; DIN (GPIO6) carries the bits. For 44.1 kHz, 16-bit stereo: BCK = 44 100 × 16 × 2 = 1.41 MHz. The DAC’s fourth clock pin (SCK) is tied to GND, which tells it to generate its internal clock from BCK.
2. **PCM5102A (digital → analog).** Turns each number into a voltage and smooths the steps. Full scale is about 2.1 V RMS. Its solder jumpers select normal I²S format and “un-mute”. TYPICAL
3. **PAM8403 (small signal → power).** A class-D amplifier: instead of making a big copy of the signal, it switches the full 5 V supply on and off about 260 000 times a second and varies the on-time. The speaker coil’s inductance averages this back into smooth current. Switching wastes little power, so it stays cool. Its gain is fixed at 24 dB, which is × 15.8. TYPICAL
4. **Speaker (current → sound).** Current through the voice coil in a magnet’s field makes a force, the cone moves, air moves. It is a geophone run backwards.

DO NOT SEND FULL VOLUME

The speaker is rated 1 W into 8 Ω. That is $V = \sqrt{(P \cdot R)} = \sqrt{8} = 2.83$ V RMS and 0.35 A. The amplifier multiplies by 15.8, so the DAC should give at most 2.83 / 15.8 = **0.18 V RMS**, which is about **21 dB below digital full scale** (roughly 9 % amplitude). Full-scale audio would drive the amplifier into hard clipping and overload the speaker. Set the level in software.

The speaker’s two wires are both live. The amplifier drives them in opposite directions (“bridge” output), which doubles the voltage swing available from 5 V. Neither wire is ground; connecting one to GND would short an output.

The modem: UART and AT commands

UART uses one wire per direction and no clock. Both sides agree on the speed in advance (115 200 bits per second) and each byte is framed so the receiver can find it:

> idle high → 1 start bit (low) → 8 data bits → 1 stop bit (high) = 10 bits per character = 87 µs

The ESP32’s transmit pin (GPIO17) goes to the modem’s receive pin and the modem’s transmit pin goes to the ESP32’s receive pin (GPIO18): **TX always goes to RX**. Swapping them is the classic mistake (it happened to us in the first test).

Voltage levels. The ESP32 speaks 3.3 V logic; the SIM868 chip speaks 2.8 V. The Waveshare board has a level-shifter chip between them. That chip needs 3.3 V from our side as a reference, which is why we feed ESP32 3.3 V into header pin 36. Without that wire the link is dead even if everything else is right.

Switching on. The modem has a power button input, PWRKEY, that must be held low for about a second. GPIO16 drives a transistor on the board that does this: GPIO16 high for 1.5 s, then the modem boots and prints `RDY`.

The conversation is plain text, one line at a time. Each command ends with a carriage return; the modem answers and finishes with `OK` or `ERROR`.

ESP32 SENDS	MODEM ANSWERS	MEANING
`AT`	`OK`	Are you there? (Also lets the modem detect our speed.)
`AT+CPIN?`	`+CPIN: READY`	SIM card is unlocked. `SIM PIN` means it still wants a PIN.
`AT+CSQ`	`+CSQ: 30,0`	Signal strength, 0–31. In dBm: −113 + 2 × 30 = −53 dBm, very strong. MEASURED
`AT+CREG?`	`+CREG: 0,1`	Registered on the home network (5 = roaming, 2 = still searching).
`ATD+36…;`	`OK`	Dial this number. The semicolon means “voice call”.
`ATH`	`OK`	Hang up.
–	`NO CARRIER`	Arrives unasked when the other side hangs up.

The radio link

- The SIM868 is a **2G (GSM)** modem. It transmits around 900 or 1800 MHz. At 900 MHz the wavelength is $\lambda = c/f = 3 \times 10^8 / 9 \times 10^8 = 33$ cm, and antennas are sized to a fraction of that.
- It transmits with up to 2 W during its 577 µs bursts (Figure 3). That peak power is where the 2 A supply bursts come from.
- **The antenna must be connected before power-on.** Without it the transmitter’s power has nowhere to go and reflects back into the chip.
- The 217 Hz burst rhythm is what causes the “dit-dit-dit” buzz near loudspeakers. Keep the antenna and the modem’s power leads away from the geophone cable and the DAC-to-amp wires.
- Both directions of a call worked on our bench on 12 Nov: outgoing, and incoming answered with `ATA`. MEASURED

7 One fall, start to finish

TIME	WHAT HAPPENS	WHERE
0 ms	A body hits the floor. The floor flexes and a wave spreads outward, mostly below 20 Hz.	floor
a few ms	The wave reaches the geophone (waves in a floor travel at hundreds of m/s). The case moves, the coil lags, a voltage of some millivolts appears.	SM-24
≤ 1 ms later	The ADC finishes its next sample, pulls DRDY low. The ESP32 clocks out 96 bits in 48 µs and checks the CRC.	ADC → SPI
+ about 15–50 ms	Filters and envelope respond; the impact envelope passes 0.3 mm/s. A hit is open.	ESP32
+ up to 0.4 s	The hit ends (envelope back under 60 % of peak). It is classed as an impact; the clock starts.	ESP32
0–1.2 s	Floor ringing is ignored.	ESP32
until the window ends	The step path keeps listening. Four steady footfalls would cancel. None come.	ESP32
window ends	FALL-LIKE. LED turns amber (30 µs of data). The voice clip streams out over I²S; the amp draws its current from the 5 V rail.	LED, DAC, amp
+ 3 s	No button press (GPIO7 stayed high). The ESP32 sends `ATD+36…;` , 17 characters, 1.5 ms on the wire. LED red.	UART
+ a few s	The modem requests a channel and starts transmitting. Supply bursts of up to 2 A; C1 and the cell deliver them.	SIM868, battery
+ about 5–10 s	The phone rings.	network

The signal path itself is fast: from impact to “hit detected” is well under a tenth of a second. Nearly all the waiting is deliberate (the window that tells a fall from a stumble) or belongs to the phone network.

Checks that follow from the physics

SENSOR

- Geophone **upright** and pressed **firmly** to the floor. It only measures vertical motion, and only what the floor passes into its case.
- Re-tune the two thresholds **on the real stage floor**. A stiff or heavy floor moves less for the same fall.
- Do a hard stomp and check the peak stays under ± 14 mm/s (no clipping).
- Keep the geophone cable twisted and away from the antenna.

POWER AND CALL

- **Fully charged cell.** A low cell plus a 2 A burst is how modems reset mid-call.
- Antenna on the “GSM” socket **before** switching on.
- **SIM PIN switched off** in a phone. In our test the PIN was still on, so the modem asked again after every power-up.
- Check `AT+CSQ` (want 10 or more) and `AT+CREG?` (want ,1 or ,5) at the venue.

8 Cheat sheets

ESP32-S3 pin map (final build)

PIN	JOB	GOES TO
GPIO1	battery voltage ÷ 2	R4/R5 midpoint
GPIO4	I²S bit clock	PCM5102A BCK
GPIO5	I²S word clock	PCM5102A LCK
GPIO6	I²S data	PCM5102A DIN
GPIO7	button (low = pressed)	lid button, R1 pull-up
GPIO8	ADC reset	Click RST
GPIO9	clock-chip select	Click CS2
GPIO10	data ready (interrupt)	Click INT
GPIO11	ADC select	Click CS1
GPIO12	SPI clock	Click SCK
GPIO13	SPI data in	Click SDO
GPIO14	SPI data out	Click SDI
GPIO15	LED data	WS2812B DIN
GPIO16	modem power key	SIM868 board pin 19
GPIO17	UART transmit	SIM868 board pin 1 (RXD)
GPIO18	UART receive	SIM868 board pin 2 (TXD)
3V3	logic supply out	Click, SIM868 pin 36, R1
5V	supply in	5 V rail

Numbers worth knowing by heart

QUANTITY	VALUE
Geophone sensitivity, with 1 kΩ	20.9 mV per mm/s
Geophone natural frequency	10 Hz
Sample rate	1000 per second
ADC resolution	24 bits, 35.8 nV per count
Measuring range (gain 4)	± 300 mV = ± 14.3 mm/s
Step band / impact band	18–60 Hz / 10–45 Hz
Step threshold	max(0.02 mm/s, 2 × background)
Impact threshold	0.3 mm/s
Footfall spacing for “walking”	0.35–1.6 s
Cancels a fall	4 steady footfalls
SPI clock / frame	2 MHz / 96 bits, 48 µs
UART to modem	115 200 bit/s, 8N1
Modem supply	3.4–4.4 V, bursts to 2 A
GSM burst	577 µs every 4.615 ms
Cell	3.6 V × 3.5 Ah = 12.6 Wh
Speaker limit	1 W = 2.83 V RMS into 8 Ω

Five questions a jury might ask

Why a geophone and not a microphone or camera?	It measures only how fast the floor moves, in a band of roughly 10–60 Hz. It cannot record speech or images, and nobody has to wear or charge anything.
Why does it need no power?	It is a generator: a magnet moving past a coil. The floor’s motion supplies the microwatts.
How do you tell a fall from a dropped object?	Not by the bang alone. An impact followed by normal walking is cancelled automatically. An impact followed by stillness triggers the spoken question, and the button cancels a false alarm before any call is made.
What if the room is noisy?	The step threshold is always at least twice the measured background, and steady hum is learned and subtracted by the adaptive filter.
2G is being switched off. Then what?	2G is still on air in Hungary. A product would use an LTE-M or NB-IoT modem; those take the same kind of AT commands over the same UART, so only the modem board changes.